

Lập trình hướng đối tượng

Java cơ bản (2)

GV: ThS. Ngô Tiến Đức

Nội dung chính

- Cấu trúc lệnh điều kiện
- Vòng lặp
- Hàm
- Một số lớp thông dụng

Cấu trúc lệnh điều kiện

Cấu trúc lệnh điều kiện với **if** và **else**

- Điều kiện (**condition**): giá trị **boolean** – đúng hoặc sai
- Câu lệnh **if**

```
if (condition) {  
    // (các) dòng lệnh/block được thực thi nếu condition đúng  
}
```

Cấu trúc lệnh điều kiện

Cấu trúc lệnh điều kiện với **if** và **else**: Câu lệnh **if - else**

```
if (condition) {  
    // block được thực thi nếu condition đúng  
} else {  
    // block được thực thi nếu condition sai  
}
```



```
if (condition1) {
    // block được thực thi nếu condition1 đúng
} else if (condition2) {
    // block được thực thi nếu condition1 sai và condition2 đúng
} else {
    // block được thực thi nếu condition2 sai (và condition1 sai)
}
```

Cấu trúc lệnh điều kiện

Cấu trúc lệnh điều kiện với **if** và **else**: Câu lệnh **if lồng (nested if)**

```
if (condition1) {  
    // block được thực thi nếu condition1 đúng  
    if (condition2) {  
        // block được thực thi nếu condition2 đúng  
    }  
}
```

Cấu trúc lệnh điều kiện

Cấu trúc lệnh điều kiện với **if** và **else**: Viết tắt (short hand)

```
variable = (condition) ? (Expression true) : (Expression False)
```

VD kiểm tra 1 số nguyên là chẵn hay lẻ:

```
// Thay vì viết:
```

```
if (a%2 == 0) System.out.println("even");
```

```
else System.out.println("odd");
```

```
// có thể viết:
```

```
String s = (a%2 == 0) ? "even" : "odd";
```

```
System.out.println(s);
```

Cấu trúc lệnh điều kiện

Cấu trúc lệnh điều kiện với **switch**:

```
switch(expression) {  
    case x:  
        // block được thực thi nếu giá trị expression == x  
        break;  
    case y:  
        // block được thực thi nếu giá trị expression == y  
        break;  
    default:  
        // block được thực thi nếu giá trị expression khác cả x và y  
}  

```


Cấu trúc lệnh điều kiện

Cấu trúc lệnh điều kiện với **switch**:

- **Expression** trong switch được tính một lần
- Giá trị của expression được so sánh với từng giá trị của từng **case** (Một case có thể chứa nhiều giá trị) và thực thi từ block có giá trị bằng với giá trị của case (**match**)
- Từ khóa **break**: kết thúc switch block
- Block trong **default** được thực thi nếu không có case nào match
- Nếu case đã match nhưng không có break?

Cấu trúc lệnh điều kiện

Cấu trúc lệnh điều kiện với **switch**: Cấu trúc cải tiến không cần **break**

```
switch(expression) {  
    case x -> { ... }  
    case y -> { ... }  
    default -> { ... }  
}
```

Vòng lặp

Vòng lặp với **for**:

```
for (statement1; statement2; statement3) {  
    // block được thực thi  
}
```

- **statement1** được thực thi một lần trước khi block được thực thi
- **statement2** xác định điều kiện để block được thực thi
- **statement3** được thực thi sau mỗi lần block được thực thi
- Biến đếm thường được khai báo trong statement1, giới hạn trong statement2 và thay đổi giá trị trong statement3

Vòng lặp với **for**: Ví dụ

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
}  
  
for (int i = 10; i > 0; i--) {  
    System.out.println(i);  
}  
  
for (int i = 0; i < 10; i+=2) {  
    System.out.println(i);  
}
```

Vòng lặp

Vòng lặp với **for**:

- Vòng lặp vô hạn: `for ( ;; ) { ... }`
- Vòng lặp lồng (**nested for**): Vòng for bên trong được thực thi đầy đủ trong mỗi lần lặp của vòng for bên ngoài

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 3; j++) {  
        System.out.println(i + " " + j);  
    }  
}
```

Vòng lặp với **for**: Vòng lặp lồng (**nested for**):

```
for (int i = 0; i < 5; i++) {  
    for (int j = 0; j <= i; j++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

Vòng lặp

Vòng lặp với **while**: Kiểm tra điều kiện và thực thi các lệnh

```
while (condition) {  
    // block được thực thi nếu condition đúng  
}
```

- Block được thực thi lặp đi lặp lại cho tới khi **condition** không còn đúng

```
int i = 1;  
while (i <= 10) {  
    System.out.println(i);  
}
```

Vòng lặp với **while**:

```
int i = 10;
while (i-- > 0) {
    System.out.println(i);
}
```

- Vòng lặp vô hạn:

```
while (true) {
    // ...
}
```

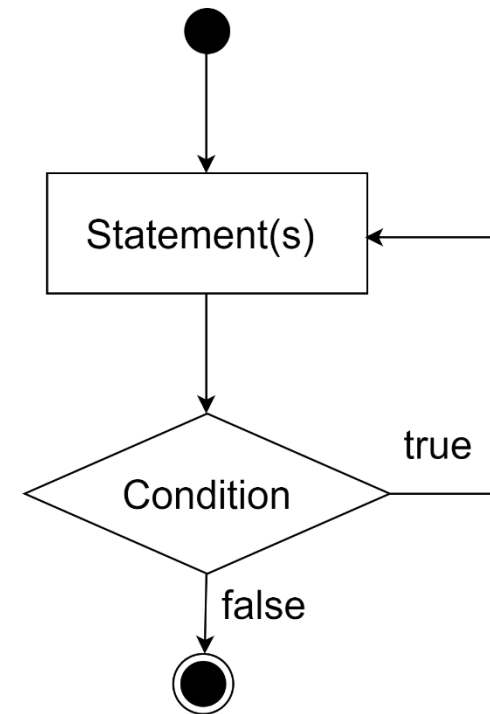
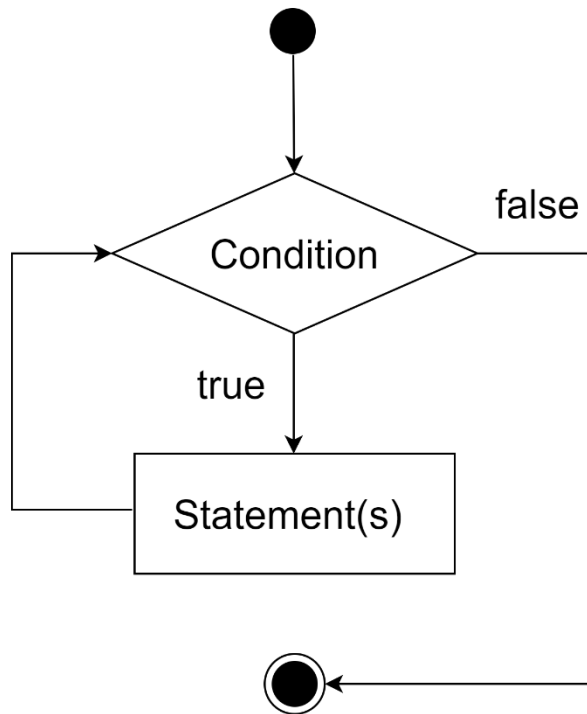

Vòng lặp

Vòng lặp với **while**:

- Vòng lặp với **do – while** thực thi các lệnh sau đó kiểm tra **condition**, nếu đúng thì lặp lại việc thực thi

```
do {  
    // block được thực thi và lặp lại nếu condition đúng  
} while (condition);
```

So sánh while và do while



Các lệnh thay đổi cấu trúc vòng lặp:

- **break:** Thoát ra khỏi vòng lặp hiện tại

```
int i = 1;
while (true) {
    if (i == 5) break;
    System.out.println(i);
    i++;
}
```

Vòng lặp

Các lệnh thay đổi cấu trúc vòng lặp:

- **continue**: Bỏ qua các câu lệnh còn lại của vòng lặp hiện tại và chuyển sang thực thi vòng lặp tiếp theo

```
for (i = 0; i < 10; i++) {  
    if (i % 2 == 0) continue;  
    System.out.println(i);  
}
```

- **Hàm/phương thức (function/method):**
 - Khởi các câu lệnh thực hiện một số công việc nhất định
 - Thực thi mỗi khi được gọi
 - Có thể truyền dữ liệu dưới dạng các tham số vào hàm
- **Tham số (parameters/params):**
 - Được định nghĩa cùng với hàm
 - Phạm vi nằm bên trong hàm
 - Được truyền giá trị khi gọi hàm

Hàm

- Khai báo hàm không có kiểu trả về:

```
public static void <function_name> (<param_type> <param_name>) {  
    <statement(s)>;  
}
```

```
public static void printSpaces(int count) {  
    for (int i = 0; i < count; i++) {  
        System.out.print(" ");  
    }  
}
```

- Gọi hàm không có kiểu trả về:

```
<function_name> (<param_value>) ;
```

```
printSpaces(10) ;
```

Hàm

- Giá trị của các tham số có kiểu dữ liệu nguyên thủy được sao chép từ giá trị của biến truyền vào

```
public static void addOne(int x) {  
    x += 1;  
}  
  
public static void main(String[] args) {  
    int x = 2;  
    addOne(x);  
    System.out.println("x = " + x); // output?  
}
```

- Khai báo hàm có kiểu trả về:

```
public static <function_type> <function_name> (<param_type> <param_name>) {  
    <statement(s)>;  
    return <return_value>;  
}
```

```
public static int addOne(int x) {  
    int y = x + 1;  
    return y;  
}  
  
System.out.println("..."); // unreachable
```


Hàm

- Đầu ra của hàm là một giá trị có kiểu được khai báo
- Hàm có kiểu trả về được gọi và sử dụng như giá trị trong các **expression**

```
int x = 2;  
x = addOne(x);  
System.out.println(x); // output?
```

```
int x = 2;  
addOne(x);  
System.out.println(x); // output?
```

Hàm

- Hàm có thể trả ra các giá trị khác nhau với những điều kiện khác nhau

```
public static int min(int a, int b) {  
    if (a > b) {  
        return b;  
    } else {  
        return a;  
    }  
    System.out.println("..."); // unreachable  
}
```

Hàm

- Phải có giá trị trả về cho tất cả các trường hợp nếu sử dụng điều kiện

```
public static int min(int a, int b) {  
    if (a > b) {  
        return b;  
    }  
} // missing return statement
```

```
public static int min(int a, int b) {  
    if (a > b) {  
        return b;  
    } else if (a <= b) {  
        return a;  
    }  
} // missing return statement
```

Hàm

- Nạp chồng phương thức (method overloading): Có thể định nghĩa 2 hàm khác nhau với cùng tên và khác tham số

```
public static int plusNumbers(int x, int y, int z) {  
    return x + y + z;  
}  
  
public static double plusNumbers(double x, double y) {  
    return x + y;  
}  
...  
int n1 = plusNumbers (1, 2, 3);  
double n2 = plusNumbers (1.2, 3.4);
```

Lớp Random

- Lớp **Random**: Cung cấp các đối tượng sinh ra các số ngẫu nhiên
- Nằm trong package `java.util.Random`;
- Khởi tạo đối tượng Random:

```
Random rand = new Random();
```

- Các phương thức của một đối tượng Random:
 - `nextInt()` : sinh ngẫu nhiên một số nguyên
 - `nextInt(max)` : sinh một số nguyên trong giới hạn `[0, max)`
 - `nextDouble()` : sinh ngẫu nhiên một số thực trong giới hạn `[0.0, 1.0)`

Lớp Random

- Sinh ngẫu nhiên một số có giá trị từ **a** đến **b**:

```
int r = rand.nextInt(b - a + 1) + a;
```

- Ví dụ:

- Sinh ngẫu nhiên một số có giá trị từ 1 đến 100:

```
int r = rand.nextInt(100) + 1;
```

- Sinh ngẫu nhiên một số có giá trị từ 10 đến 100:

```
int r = rand.nextInt(91) + 10;
```

- Sinh ngẫu nhiên một số có giá trị từ 0 đến 100:

```
int r = rand.nextInt(101);
```

Lớp Math

- Lớp **Math**: Cung cấp các phương thức hỗ trợ thực hiện các phép tính toán học (mathematical calculations)
- Nằm trong package `java.lang`
- Cú pháp gọi: `Math.<method_name> (<parameter(s)>)`
- VD:

```
double squareRoot = Math.sqrt(25.0); // 5.0
```

```
int absoluteValue = Math.abs(-10); // 10
```

```
int minValue = Math.min(1, 2) // 1
```

Lớp Math

Một số phương thức của lớp Math

Tên	Mô tả
abs(value)	Giá trị tuyệt đối
sqrt(value)	Căn bậc 2
log(value)	Logarit cơ số e
log10(value)	Logarit cơ số 10
max(value1, value2)	Giá trị lớn hơn trong 2 số
min(value1, value2)	Giá trị nhỏ hơn trong 2 số
pow(base, exponent)	Lũy thừa
ceil(value)	Làm tròn lên số nguyên gần nhất
floor(value)	Làm tròn xuống số nguyên gần nhất
round(value)	Làm tròn lên hoặc xuống tới số nguyên gần nhất

Luyện tập

- Bài 1: Viết chương trình in ra số Fibonacci thứ n ($n > 0$).
- Bài 2: Viết chương trình nhập vào một số nguyên dương và kiểm tra xem có phải là số nguyên tố hay không.
- Bài 3: Viết chương trình mô phỏng hành động tung 2 chiếc xúc xắc cho tới khi tổng kết quả của cả 2 là 7.

Sample run:

Roll: 2 + 4 = 6

Roll: 3 + 5 = 8

Roll: 4 + 3 = 7

You won after 3 tries!

Luyện tập

```
// Bài 1  
  
public static int fibonacci(int n) {  
    if(n == 0) return 0;  
    else if(n == 1) return 1;  
    else return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

```
// Bài 2
public static boolean isPrimeNumber(int n) {
    if (n < 2)    return false;
    for (int i = 2; i <= Math.sqrt(n); i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

```
// Bài 3
Random rand = new Random();
int sum = 0;
int tries = 0;
while (sum != 7) {
    int roll1 = rand.nextInt(6) + 1;
    int roll2 = rand.nextInt(6) + 1;
    sum = roll1 + roll2;
    System.out.println("Roll: " + roll1 + " + " + roll2 + " = " + sum);
    tries++;
}
System.out.println("You won after " + tries + " tries!");
```

Lớp bao

- Lớp bao (**Wrapper classes**): Cung cấp các phương thức để thao tác với kiểu dữ liệu nguyên thủy dưới dạng các đối tượng
- Nằm trong package `java.lang`
- VD:

```
Double n = 5.5;
```

```
System.out.println(n); // 5.5
```

```
System.out.println(n.doubleValue()); // 5.5
```

```
System.out.println(n.intValue()); // 5
```

Lớp bao

Primitive Data Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

Lớp bao

- Một số phương thức của lớp bao (ví dụ **Integer**):
 - `Integer.parseInt("<string_value>")`: Chuyển từ chuỗi ký tự sang số nguyên (nếu hợp lệ)
 - `Integer.sum(<value1>, <value 2>)`: Tổng 2 số
 - `Integer.MAX_VALUE`, `Integer.MIN_VALUE`: Giá trị integer lớn nhất và nhỏ nhất
 - `<integer_value>.toString()`: Chuyển số nguyên sang dạng chuỗi ký tự

Lớp BigInteger

- Lớp **Big Integer**: Sử dụng cho các phép tính toán với số nguyên rất lớn vượt ngoài giá trị của các kiểu dữ liệu nguyên thủy có sẵn
- Nằm trong package `java.math.BigInteger`;
- Khởi tạo và gán giá trị cho số BigInteger:
 - `BigInteger b = BigInteger.ZERO; // (ONE/TWO/TEN)`
 - `BigInteger b = BigInteger.valueOf(<long_value>);`
 - `BigInteger b = new BigInteger("<string_value>");`

Lớp BigInteger

- Một số phương thức của lớp BigInteger:
 - `a.add(b)`: Tính tổng a và b
 - Tương tự với `subtract`, `multiply`, `divide`, `mod`, `sqrt`,...
 - `a.equals(b)`: Kiểm tra a có bằng b không, trả về giá trị boolean
 - `a.compareTo(b)`: So sánh a và b, trả về -1 nếu a nhỏ hơn, 0 nếu bằng nhau và 1 nếu a lớn hơn



- 42